# Service

Цели работы:

- изучить типы Service в Kubernetes
- освоить создание ClusterIP для внутреннего доступа
- научиться настраивать NodePort для внешнего доступа
- понять принципы связывания Service с Pod'ами

**Service (Сервис)** — это абстракция в Kubernetes, которая предоставляет стабильную точку (endpoint) для доступа к группе динамических Pod'ов.

**Основные проблемы, которые решает Service:**

1. **Динамичность Pod'ов**:
   - они могут пересоздаваться, масштабироваться, перемещаться между узлами
   - при каждом пересоздании Pod получает новый IP-адрес
   - Service предоставляет постоянный IP-адрес, не зависящий от жизненного цикла отдельных Pod'ов
2. **Балансировка нагрузки**:
   - Когда у Deployment'а несколько реплик Pod'ов, Service автоматически распределяет трафик между ними
   - Обеспечивает высокую доступность приложения
3. **Сетевая абстракция**:
   - Service скрывает внутреннюю структуру приложения от клиентов
   - Клиенты обращаются к единой точке входа, не зная о количестве и расположении Pod'ов

## Типы Service

*ClusterIP (Тип по умолчанию)*

- **Назначение**: Внутренний доступ в пределах кластера
- **Видимость**: только внутри кластера Kubernetes
- **IP-адрес**: Виртуальный IP из диапазона служб кластера
- **Использование**: Связь между микросервисами внутри кластера
- **Пример**: Сервис базы данных, доступный только для внутренних сервисов

```yaml
spec:
  type: ClusterIP  # можно не указывать, так как это тип по умолчанию
```

*NodePort*

- **Назначение**: Внешний доступ к приложению
- **Видимость**: Доступен извне кластера через порты на узлах
- **Порты**: Открывает порт в диапазоне 30000-32767 на КАЖДОМ узле кластера
- **Цепочка доступа**: Внешний клиент → NodeIP:NodePort → Service → Pod
- **Использование**: Разработка, тестирование, простые production-среды

```yaml
spec:
  type: NodePort
  ports:
```

```
  - nodePort: 31000  # опционально (иначе будет выбран случайный)
```

*LoadBalancer*

- **Назначение**: публичный доступ в cloud-средах
- **Видимость**: внешний IP-адрес, доступный из интернета
- **Особенности**: автоматически создает cloud-балансировщик нагрузки (AWS ELB, GCP Load Balancer, etc.)
- **Использование**: Production-приложения в cloud-провайдерах

```
spec:
  type: LoadBalancer
```

## Принцип работы селекторов и endpoints

*Селекторы (Selectors)*

- **Назначение**: определяют, какие Pod'ы должны быть включены в Service
- **Механизм**: Service ищет Pod'ы с метками (labels), совпадающими с селектором
- **Синтаксис**: Пары ключ-значение в формате `key: value`

```
# Service определяет селектор
spec:
  selector:
    app: frontend
    tier: web
# Pod должен иметь соответствующие метки
metadata:
  labels:
    app: frontend
    tier: web
```

*Endpoints*

- **Определение**: Динамический список IP-адресов и портов Pod'ов, соответствующих селектору
- **Автоматическое обновление**: Kubernetes автоматически обновляет endpoints при:
  - Создании новых Pod'ов
  - Удалении Pod'ов
  - Изменении меток Pod'ов
- **Просмотр**: `kubectl get endpoints <service-name>`

### Пример работы:

Service "web-service" (IP: 10.96.105.45)

↓

Selector: app=web

↓

Endpoints: 10.244.1.5:80, 10.244.2.3:80, 10.244.3.7:80

(три Pod'а, соответствующих селектору)

# Объяснение портов: port vs targetPort vs nodePort

*Трехуровневая система портов:*

```yaml
ports:
- name: http
  protocol: TCP
  port: 80           # ← Port (обязательный)
  targetPort: 8080   # ← TargetPort (обязательный для ClusterIP)
  nodePort: 30080    # ← NodePort (только для NodePort/LoadBalancer)
```

*Port (порт сервиса)*

- **Назначение**: Порт, на котором Service принимает запросы ВНУТРИ кластера
- **Видимость**: только для других компонентов кластера
- **Пример**: Другой Service обращается к web-service:80

*TargetPort (порт пода)*

- **Назначение**: Порт контейнера, на который Service перенаправляет трафик
- **Должен совпадать**: С containerPort в спецификации Pod'a
- **Пример**: Приложение в контейнере слушает порт 8080

*NodePort (порт узла)*

- **Назначение**: Порт на КАЖДОМ узле кластера для внешнего доступа
- **Диапазон**: 30000-32767 (если не указан, выбирается случайно)
- **Пример**: Внешний клиент обращается к http://node-ip:30080

*Визуализация потоков данных:*

text

# ClusterIP Service:

[Client Pod] → Service:Port (80) → Pod:TargetPort (8080)

# NodePort Service:

[External Client] → NodeIP:NodePort (30080) → Service:Port (80) → Pod:TargetPort (8080)

*Важные замечания:*

- Если targetPort не указан, используется тот же порт, что и port
- nodePort должен быть уникальным в пределах кластера
- Изменение портов требует обновления Service (нельзя изменить порты у существующего Service)

## Пример 1. Базовый ClusterIP Service

Создание Deployment с веб-сервером и ClusterIP Service для внутреннего доступа внутри кластера. Проверка работы сервиса через port-forward и анализ назначенных IP-адресов и endpoints. (*На практике используется для доступа к базам данных, внутренним API, бэкенд-сервисам, которые не должны быть доступны извне*).

**Шаг 1.** Создаем Deployment

*Замечание! Если у вас остался Deployment web-app из предыдущих ЛР, тот его нужно удалить*

deployment-clusterip.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
        environment: test
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.25-alpine
        ports:
        - containerPort: 80
        env:
        - name: NGINX_PORT
          value: "80"
```

Применим Deployment и проверим созданные Pod'ы:

```
C:\Temp>kubectl apply -f deployment-clusterip.yaml
deployment.apps/web-app created

C:\Temp>kubectl get pods -l app=web-app
NAME                       READY   STATUS    RESTARTS   AGE
web-app-8449965dd9-bsbmx   1/1     Running   0          116s
web-app-8449965dd9-cmh8q   1/1     Running   0          116s
web-app-8449965dd9-p6kfg   1/1     Running   0          116s
```

**Шаг 2.** Создаем ClusterIP Service

service-clusterip.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
  labels:
    service-type: internal
spec:
  type: ClusterIP
  selector:
    app: web-app  # Должен совпадать с labels Pod'ов
  ports:
  - name: http
    protocol: TCP
    port: 80        # Порт сервиса
    targetPort: 80  # Порт контейнера (должен совпадать с containerPort)
```

Применим Service

```
C:\Temp>kubectl apply -f service-clusterip.yaml
service/web-service created
```

**Шаг 3.** Проверяем работу

Проверка Service:

```
C:\Temp>kubectl get service web-service
NAME          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
web-service   ClusterIP   10.98.121.220   <none>        80/TCP    109s
```

Проверка Endpoints. Видим IP-адреса всех Pod'ов:

```
C:\Temp>kubectl get endpoints web-service
NAME          ENDPOINTS                                   AGE
web-service   10.1.0.48:80,10.1.0.49:80,10.1.0.50:80   2m30s
```

Тестирование изнутри кластера:

Создадим тестовый Pod для проверки доступа:

```
kubectl run test-client --image=busybox --rm -it --restart=Never -- sh
```

Внутри контейнера выполним:

# Используем DNS имя сервиса

```
wget -q -O- http://web-service.default.svc.cluster.local
```

```
C:\Temp>kubectl run test-client --image=busybox --rm -it --restart=Never -- sh
If you don't see a command prompt, try pressing enter.
/ # wget -q -O- http://web-service.default.svc.cluster.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ #
```

# Или короткая форма (в пределах того же namespace)

wget -q -O- http://web-service

```
/ # wget -q -O- http://web-service
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ #
```

Альтернативная проверка с port-forward:

```
C:\Temp>kubectl port-forward service/web-service 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

Теперь откройте в браузере: http://localhost:8080

**Шаг 4.** Выводы

- Service получил стабильный ClusterIP (например, 10.98.121.220)
- Endpoints автоматически включили все Pod'ы с меткой app: web-app
- DNS имя сервиса разрешается в ClusterIP
- Балансировка работает - разные запросы могут попадать на разные Pod'ы

## Пример 2. NodePort для внешнего доступа

Настройка NodePort Service для обеспечения доступа к приложению извне кластера. Тестирование доступности через порты узлов и проверка балансировки нагрузки между Pod'ами (*Настройка внешнего доступа к веб-приложениям, API или интерфейсам управления через фиксированные порты на узлах кластера. Применяется в тестовых средах, демонстрационных стендах или когда cloud load balancer недоступен. Похожий подход используется для доступа к веб-интерфейсам панелей управления, мониторинга или временного доступа к сервисам*)

**Шаг 1.** Создаем Deployment. Можно использовать тот же Deployment из Примера 1, либо создать новый.

**Шаг 2.** Создаем NodePort Service

service-nodeport.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
  labels:
    service-type: external
spec:
  type: NodePort
  selector:
    app: web-app  # Связываем с нашими Pod'ами
  ports:
  - name: http
    protocol: TCP
    port: 80            # Порт сервиса (внутри кластера)
    targetPort: 80      # Порт контейнера
    nodePort: 30080     # Порт на узлах (30000-32767)
```

Применим Service

**Шаг 3.** Проверяем работу

Проверка Service:

Определяем IP-адреса узлов:

```
C:\Temp>kubectl get service web-nodeport
NAME          TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
web-nodeport  NodePort   10.96.222.106   <none>        80:30080/TCP   94s

C:\Temp>kubectl get nodes -o wide
NAME             STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE         KERNEL-VERSION                     CONTAINER-RUNTIME
docker-desktop   Ready    control-plane   14d   v1.32.2   192.168.65.3   <none>        Docker Desktop   5.15.153.1-microsoft-standard-WSL2   docker://28.4.0
```

Тестирование доступа:

а) Локальное тестирование:

```
C:\Temp>curl http://localhost:30080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

б) Использование port-forward (если нет прямого доступа к узлам):

kubectl port-forward service/web-nodeport 8080:80

Затем в браузере: http://localhost:8080

**Шаг 4.** Проверка балансировки

Создадим скрипт для проверки балансировки нагрузки:

Linux

```
# test-balancing.sh
for i in {1..10}; do
  echo "Request $i:"
  curl -s http://NODE_IP:30080 | grep "Welcome to nginx" |
head -1
done
```

windows

```
@echo off
for /L %%i in (1,1,10) do (
    echo Request %%i:
    curl -s http://localhost:30080 | findstr "Welcome to
nginx"
    timeout 1 > nul
)
```

Запустим скрипт и увидим, как запросы распределяются между разными Pod'ами.

```
C:\Temp>test-balancing.bat
Request 1:
<title>Welcome to nginx!</title>
body { width: 35em; margin: 0 auto;
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
<a href="http://nginx.com/">nginx.com</a>.</p>
<p><em>Thank you for using nginx.</em></p>
Request 2:
<title>Welcome to nginx!</title>
body { width: 35em; margin: 0 auto;
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
<a href="http://nginx.com/">nginx.com</a>.</p>
<p><em>Thank you for using nginx.</em></p>
Request 3:
<title>Welcome to nginx!</title>
body { width: 35em; margin: 0 auto;
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
<a href="http://nginx.com/">nginx.com</a>.</p>
<p><em>Thank you for using nginx.</em></p>
Request 4:
```

Создадим еще один NodePort Service без указания nodePort:

service-nodeport-auto.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport-auto
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
    # nodePort не указан - будет выбран автоматически
```

Проверим, какой порт был назначен:

После выполнения обоих примеров сравним сервисы:

```
C:\Temp>kubectl apply -f service-nodeport-auto.yaml
service/web-nodeport-auto created

C:\Temp>kubectl get service web-nodeport-auto
NAME                TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
web-nodeport-auto   NodePort   10.110.51.191   <none>        80:31973/TCP   12s

C:\Temp>kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP        14d
web-nodeport        NodePort    10.96.222.106   <none>        80:30080/TCP   23m
web-nodeport-auto   NodePort    10.110.51.191   <none>        80:31973/TCP   19s
web-service         ClusterIP   10.98.121.220   <none>        80/TCP         38m
```

**Отличия:**

**ClusterIP**: Доступен только внутри кластера по стабильному IP
**NodePort**: Доступен извне через порты на узлах, сохраняет внутренний ClusterIP

**Выводы**

- Сервис получил стабильный ClusterIP и внешний NodePort в диапазоне 30000-32767
- Endpoints автоматически включили все Pod'ы с меткой app: web-app
- Балансировка нагрузки подтверждена - запросы распределяются между разными Pod'ами

## Пример 3. Service для многоконтейнерного Pod

Развертывание Pod с двумя контейнерами и создание отдельных сервисов для каждого из них. Проверка доступа к разным контейнерам через внутренние и внешние точки входа (*Организация доступа к разным сервисам, работающим в одном Pod - например, веб-серверу и API, основному приложению и sidecar-контейнеру с дополнительной функциональностью. Используется в сценариях, когда контейнеры логически связаны, но предоставляют разные endpoints, как в микросервисных архитектурах с совместно размещенными компонентами.*)

**Шаг 1.** Создаем многоконтейнерный Deployment

***deployment-multi.yaml***

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: multi-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: multi-app
  template:
    metadata:
      labels:
        app: multi-app
    spec:
      containers:
     # Контейнер 1: Nginx (веб-сервер)
      - name: nginx-web
        image: nginx:1.25-alpine
        ports:
        - containerPort: 80

     # Контейнер 2: Apache с правильной конфигурацией
      - name: apache-api
        image: httpd:2.4-alpine
        ports:
        - containerPort: 8080
        command: ["/bin/sh"]
        args:
          - "-c"
          - |
            # Копируем стандартную конфигурацию и меняем порт
            cp /usr/local/apache2/conf/httpd.conf
/usr/local/apache2/conf/httpd-custom.conf
            sed -i 's/^Listen 80/Listen 8080/g'
/usr/local/apache2/conf/httpd-custom.conf
            sed -i 's/^#ServerName www.example.com:80/ServerName
localhost:8080/g' /usr/local/apache2/conf/httpd-custom.conf

            # Создаем тестовую страницу
```

```
            echo "<h1>Apache Server working on port 8080</h1>" >
/usr/local/apache2/htdocs/index.html
            echo "<p>Pod: $(hostname)</p>" >>
/usr/local/apache2/htdocs/index.html

            # Запускаем
            httpd -f /usr/local/apache2/conf/httpd-custom.conf -DFOREGROUND
```

Применим Deployment:

Проверим созданные Pod'ы:

```
C:\Temp>kubectl get pods -l app=multi-app
NAME                          READY   STATUS              RESTARTS   AGE
multi-app-54d6899b49-j7nqt    0/2     ContainerCreating   0          5s
multi-app-54d6899b49-qzmv7    0/2     ContainerCreating   0          5s

C:\Temp>kubectl get pods -l app=multi-app
NAME                          READY   STATUS    RESTARTS   AGE
multi-app-54d6899b49-j7nqt    2/2     Running   0          11s
multi-app-54d6899b49-qzmv7    2/2     Running   0          11s
```

**Шаг 2.** Создаем отдельные сервисы для каждого контейнера

service-multi.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: multi-app  # Связываем с нашими Pod'ами
  ports:
  - name: nginx-port
    protocol: TCP
    port: 80           # Порт сервиса для nginx
    targetPort: 80     # Порт контейнера nginx

---
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  type: ClusterIP
  selector:
    app: multi-app  # Тот же селектор - оба сервиса указывают на одни Pod'ы
  ports:
  - name: apache-port
    protocol: TCP
    port: 8080         # Порт сервиса для apache
    targetPort: 8080   # Порт контейнера apache
```

Применим сервисы

**Шаг 3.** Создаем NodePort Service для внешнего доступа

service-multi-nodeport.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: Service
metadata:
  name: multi-app-external
spec:
  type: NodePort
  selector:
    app: multi-app
  ports:
  - name: web-external
    protocol: TCP
    port: 80            # Порт сервиса
    targetPort: 80      # Порт nginx контейнера
    nodePort: 30081     # Внешний порт для nginx

  - name: api-external
    protocol: TCP
    port: 8080          # Порт сервиса
    targetPort: 8080    # Порт apache контейнера
    nodePort: 30082     # Внешний порт для apache
```

Применим сервисы

**Шаг 4.** Проверим работу

1) *Проверка всех сервисов:*

```
C:\Temp>kubectl get service
NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                    AGE
api-service          ClusterIP   10.101.173.175   <none>        8080/TCP                   26m
kubernetes           ClusterIP   10.96.0.1        <none>        443/TCP                    14d
multi-app-external   NodePort    10.111.130.8     <none>        80:30081/TCP,8080:30082/TCP 25m
web-nodeport         NodePort    10.96.222.106    <none>        80:30080/TCP               58m
web-nodeport-auto    NodePort    10.110.51.191    <none>        80:31973/TCP               35m
web-service          ClusterIP   10.98.121.220    <none>        80/TCP                     73m
```

### 2) *Тестирование внутренних сервисов:*

- запускаем тестовый контейнер
- тестируем nginx сервис
- тестируем apache сервис
- выходим из контейнера

```
C:\Temp>kubectl run test-client --image=busybox --rm -it --restart=Never -- sh
If you don't see a command prompt, try pressing enter.
/ # wget -q -O - http://web-service:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ # wget -q -O - http://api-service:8080
<h1>Apache Server working on port 8080</h1>
<p>Pod: multi-app-54d6899b49-qzmv7</p>
/ # exit
```

3) *Тестирование внешнего доступа:*

```
C:\Temp>curl http://localhost:30081
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

```
C:\Temp>curl http://localhost:30082
<h1>Apache Server working on port 8080</h1>
<p>Pod: multi-app-54d6899b49-j7nqt</p>
```

**Шаг 5.** Проверка сетевой изоляции

Убедимся, что контейнеры внутри Pod'а могут общаться друг с другом через localhost:

- подключаемся к одному из pod'ов
- из контейнера nginx проверяем доступность apache
- выходим

```
C:\Temp>kubectl get pods
NAME                              READY   STATUS    RESTARTS        AGE
app-2                             2/2     Running   4 (6d23h ago)   13d
hello                             1/1     Running   2 (6d23h ago)   13d
multi-app-54d6899b49-j7nqt        2/2     Running   0               6m53s
multi-app-54d6899b49-qzmv7        2/2     Running   0               6m53s
recreate-app-78dc7c4f59-k4qhw     1/1     Running   1 (6d23h ago)   6d23h
recreate-app-78dc7c4f59-pg2c8     1/1     Running   1 (6d23h ago)   6d23h
recreate-app-78dc7c4f59-sjmd8     1/1     Running   1 (6d23h ago)   6d23h
rolling-app-5867bd6975-ks2hv      1/1     Running   1 (6d23h ago)   6d23h
rolling-app-5867bd6975-pd2tb      1/1     Running   1 (6d23h ago)   6d23h
rolling-app-5867bd6975-sk6wx      1/1     Running   1 (6d23h ago)   6d23h
web-app-8449965dd9-bsbmx          1/1     Running   0               83m
web-app-8449965dd9-cmh8q          1/1     Running   0               83m
web-app-8449965dd9-p6kfg          1/1     Running   0               83m

C:\Temp>kubectl exec -it multi-app-54d6899b49-j7nqt -c nginx-web -- sh
/ # wget -q -O- http://localhost:8080
<h1>Apache Server working on port 8080</h1>
<p>Pod: multi-app-54d6899b49-j7nqt</p>
/ # exit
```

**Выводы**

- Созданы два отдельных сервиса, указывающие на одни Pod'ы, но на разные контейнеры
- Frontend-service направляет трафик на порт 80 nginx, backend-service - на порт 8080 apache
- Endpoints содержат одинаковые IP-адреса Pod'ов с разными портами целевых контейнеров
- Внешний multi-app-external сервис предоставляет доступ к обоим контейнерам через разные NodePort

## *Задания для самостоятельного решения*

***Задание 5.1***: Базовое управление Service
*Цель*: закрепить навыки работы с командами описания и диагностики Service.

*Задача:*
Создайте Deployment с именем web-app, который использует образ nginx:1.25 и имеет 3 реплики. Затем создайте ClusterIP Service с именем web-service для доступа к этому Deployment.

Выполните следующие команды и проанализируйте их вывод:

```
kubectl get service web-service -o wide
kubectl describe service web-service
kubectl get endpoints web-service
kubectl get pods -l app=web-app --show-labels
```

*Вопросы для отчета:*
Какой ClusterIP был назначен сервису?
Сколько endpoints связано с сервисом и какие у них адреса?
Какой селектор использует Service для поиска Pod'ов?
Какой тип сервиса используется по умолчанию?

***Задание 5.2.*** Создание и тестирование NodePort
*Цель*: освоить создание сервисов для внешнего доступа и их тестирование.

*Задача*:
На основе Deployment из задания 5.1 создайте NodePort Service с именем web-nodeport, который использует порт 30090 на узлах.

Выполните последовательность действий:
- создайте NodePort Service
- проверьте назначенные порты
- протестируйте доступность сервиса:
  - определите IP узла
  - протестируйте доступность
  - проверьте работу через port-forward:

*Вопросы для отчета:*
Какой NodePort был назначен сервису?
Доступен ли сервис извне кластера?
Какая разница между доступом через NodePort и port-forward?
Как проверить, что трафик балансируется между разными Pod'ами?

**Задание 5.3.** Многоконтейнерный Service
*Цель*: научиться работать с сервисами для сложных приложений с несколькими контейнерами.

*Задача:* Создайте Deployment multi-service-app со следующими характеристиками:
- 2 реплики
- Два контейнера в каждом Pod:
  - frontend: образ nginx:1.25-alpine, порт 80
  - backend: образ httpd:2.4-alpine, порт 8080
- Создайте два отдельных ClusterIP Service:
  - frontend-service для доступа к nginx на порту 80
  - backend-service для доступа к apache на порту 8080

*Проверочные команды:*
```
kubectl get services -l app=multi-service-app
kubectl describe service frontend-service
kubectl describe service backend-service
kubectl get endpoints
```

*Вопросы для отчета:*
Могут ли два сервиса указывать на одни и те же Pod'ы?
Как проверить, что каждый сервис перенаправляет трафик на правильный контейнер?
Сколько всего endpoints создано для этого Deployment?
Как организовать доступ к обоим сервисам извне кластера?

***Задание 5.4.*** Диагностика
*Цель*: научиться диагностировать и устранять типичные проблемы с сервисами.

*Задача:* Создайте проблемные сценарии и исправьте их:

*Сценарий А:* Сервис без matching Pod'ов

Создайте Service с селектором app: non-existent
Диагностируйте проблему:
```
kubectl get endpoints <service-name>
kubectl describe service <service-name>
```
Исправьте проблему

*Сценарий Б:* Неправильные порты
Создайте Pod, который слушает на порту 8080

Создайте Service с targetPort: 80
Диагностируйте и исправьте проблему

*Вопросы для отчета:*
Какие признаки у сервиса, который не нашел Pod'ы?
Как быстро определить проблему с неправильными портами?
Какие команды наиболее эффективны для диагностики сетевых проблем?
Как проверить, что DNS работает корректно внутри кластера?

**Задание 5.5.** Сервисы для рабочей среды
*Цель*: научиться создавать сервисы для production-среды.

*Задача*:

Создайте набор сервисов для production-приложения со следующими требованиями:
- Deployment production-frontend: 3 реплики, образ nginx:1.25, порт 80
- Deployment production-backend: 2 реплики, образ httpd:2.4, порт 8080
- Service frontend-service: ClusterIP для внутреннего доступа
- Service frontend-external: NodePort для внешнего доступа (порт 30080)
- Service backend-service: ClusterIP только для внутреннего использования

Проверочные команды:
```
kubectl get all -l environment=production
kubectl describe service frontend-external
kubectl get endpoints --selector=environment=production
kubectl exec -it <pod-name> -- curl http://backend-service:8080
```

Дополнительные требования:
- все ресурсы должны иметь метку environment: production
- сервисы должны использовать понятные имена портов
- backend сервис не должен быть доступен извне кластера

*Вопросы для отчета:*
Как организовать сетевую изоляцию между frontend и backend?
Как проверить, что backend доступен только изнутри кластера?
Как проверить, что backend доступен только изнутри кластера?

***Задание 5.6***: Мониторинг и диагностика сервисов
*Цель*: освоить инструменты мониторинга состояния сервисов.

*Задача*: Для созданных сервисов выполните комплексную диагностику:

Базовый мониторинг:

```
kubectl get services --show-labels
kubectl get endpoints
kubectl describe service frontend-service
```

Проверка DNS:

```
kubectl run dns-check --image=busybox --rm -it --
restart=Never -- nslookup frontend-service
```

Логи и события:

```
kubectl logs deployment/production-frontend --tail=10
kubectl get events --field-selector
involvedObject.kind=Service
```

*Вопросы для отчета:*

Какие метрики важны для мониторинга сервисов?
Как определить, что сервис работает корректно?
Какие инструменты наиболее эффективны для диагностики сетевых проблем?

***Контрольные вопросы для всех заданий***

В чем основное отличие ClusterIP от NodePort?
Как Service находит Pod'ы для балансировки нагрузки?

Когда следует использовать несколько сервисов для одного Deployment?
Как организовать доступ к разным контейнерам в одном Pod?
Какие ограничения есть у NodePort сервисов?

Какие три команды вы бы использовали для быстрой диагностики проблемного сервиса?
Как определить, что проблема с сетью, а не с приложением?
Какие логи наиболее полезны при проблемах с сервисами?